

Avance de Trabajo Final 3

Gestor de biblioteca

Integrantes:

- Tapia Santa Adriana Lucia
- Salazar Camacho Sandro Piero
- Rojas Gamonal Angel Samir
- Ramos Valdivia Gerardo Adrian

Contenido

01 **Introducción**

02 **Arquitectura y desarrollo del proyecto**

03 **Desarrollo del proyecto**

04 **Flujo de trabajo Git y GitHub**

05 **Integración Continua (CI)**

06 **Calidad y Seguridad**

07 **Dockerización y Pipeline CI/CD**

08 **Jira**

09 **Jira Service Management**

10 **Incidentes y Bugs**

11 **Automatización Jira-GitHub**

12 **Conclusiones**

Realidad problemática

A pesar del aumento en la lectura digital en el Perú, existe una crisis de retención que genera un 0% de ahorro de reaprendizaje debido a la falta de herramientas de síntesis. Para superar la dispersión de las aplicaciones de notas y la superficialidad de las redes sociales de libros, se requiere urgentemente una solución que centralice la organización bibliográfica y potencie el análisis crítico.

Objetivo general

Diseñar e implementar un software de gestión de libros favoritos que integre una API externa para organizar lecturas, almacenar resúmenes y opiniones personales, con el fin de fortalecer la memoria crítica y la gestión del conocimiento en la era digital.



Metodología

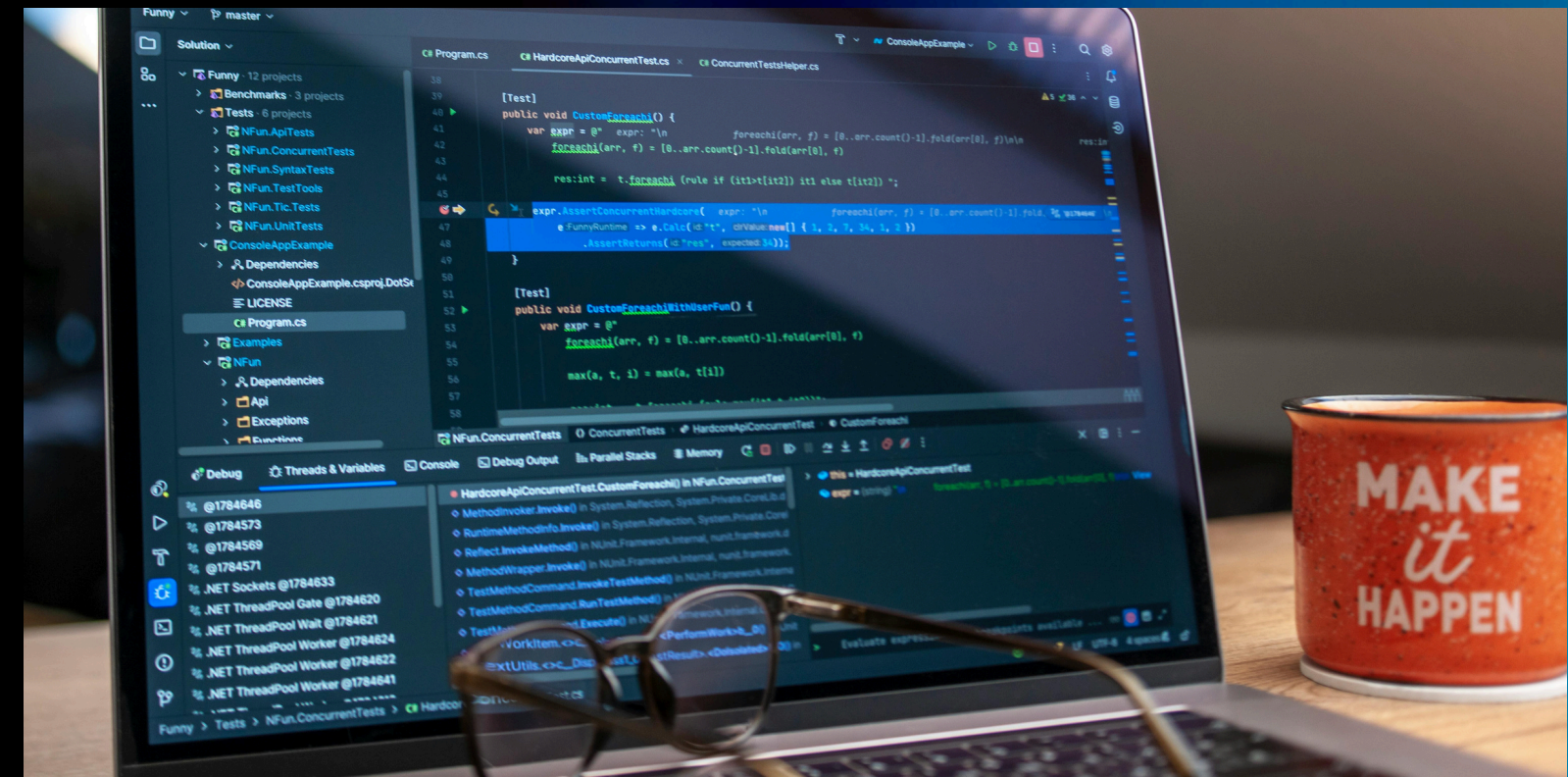
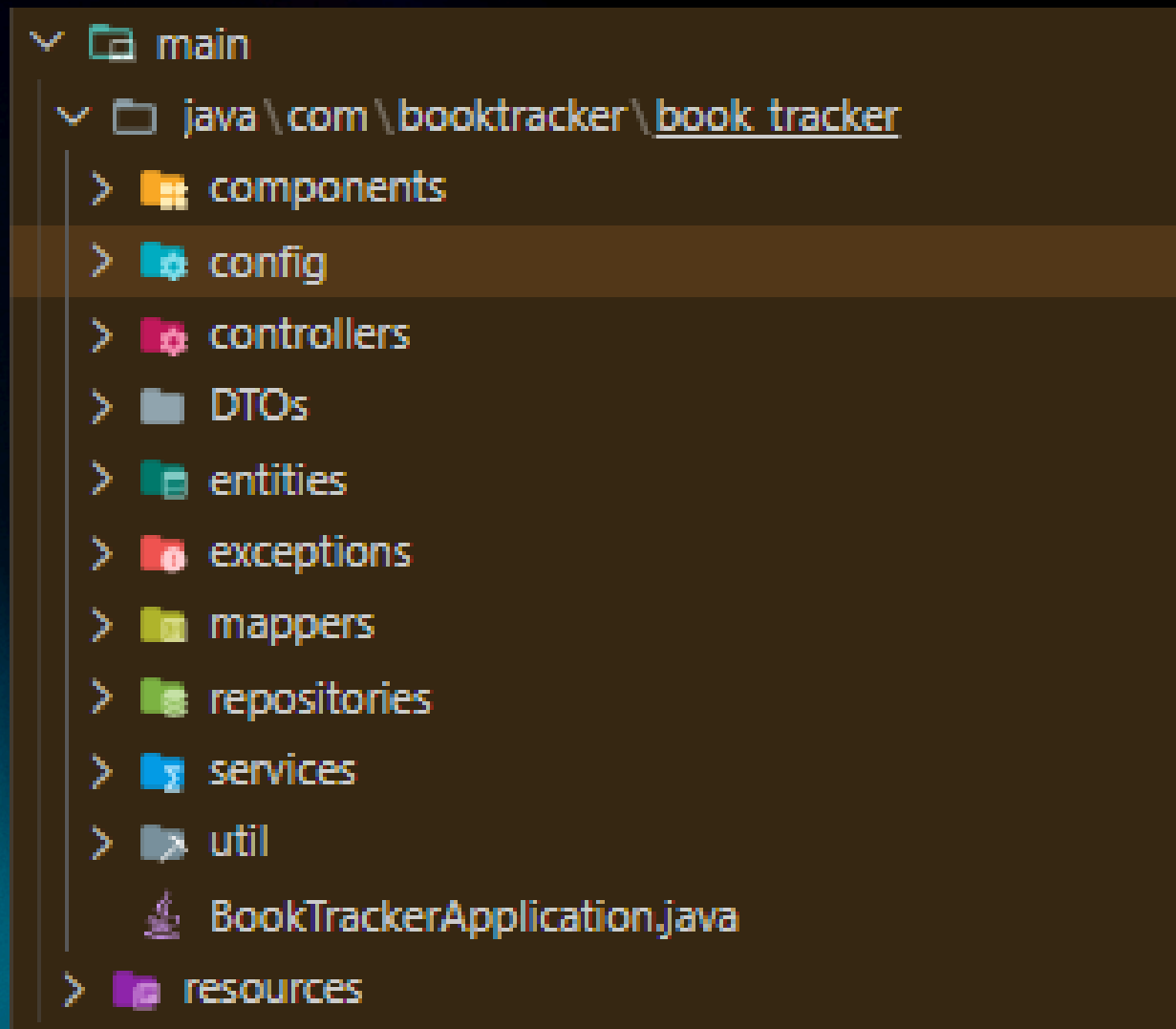
El proyecto consiste en una investigación aplicada con enfoque cualitativo, orientada a desarrollar una solución tecnológica que resuelva la deficiencia en la retención de información digital. Se fundamenta en el análisis del comportamiento del usuario y la revisión documental para diseñar una herramienta que gestione eficientemente el conocimiento personal.

Arquitectura y desarrollo del proyecto

Arquitectura del sistema

Arquitectura en capas

La aplicación utiliza una arquitectura en capas para separar responsabilidades y facilitar el mantenimiento del sistema.



Componentes

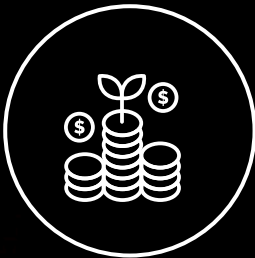
- Controllers
- Services
- Repositories
- Entities
- DTOs

Estrategia de Persistencia y Base de Datos

Uso de PostgreSQL con Spring Data JPA e Hibernate para una gestión relacional eficiente.

Desarrollo de trabajo

Flujo de Trabajo (Git/GitHub): Control de versiones profesional mediante el uso de ramas (branches) y commits descriptivos, facilitando la integración de nuevas funcionalidades construidas.



Módulos implementados

- Módulo de autenticación
- Módulo de libro
- Módulo de listas
- Módulo de reseñas



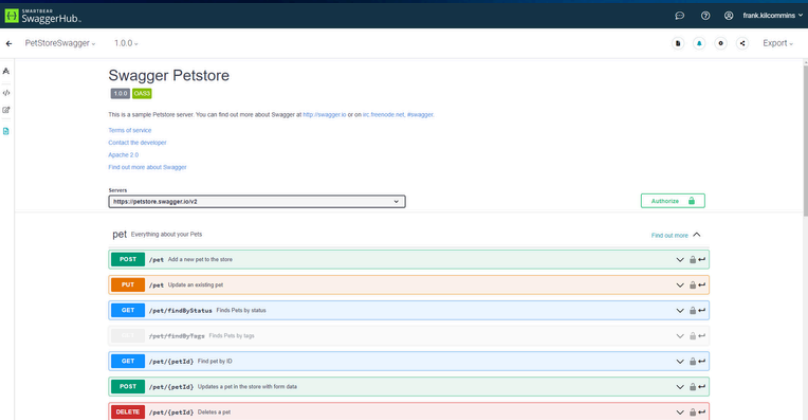
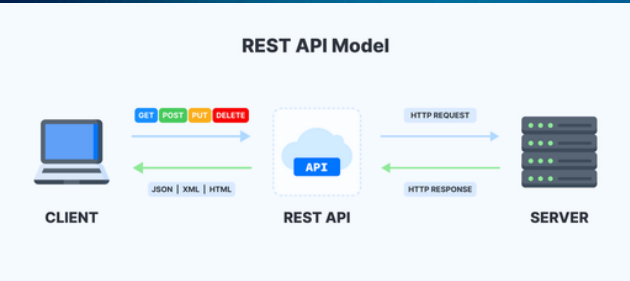
API Rest

Endpoints implementados. Los endpoints REST permiten realizar operaciones CRUD utilizando respuestas JSON.



Swagger

Se integró Swagger para documentar automáticamente todos los endpoints REST.



Herramienta Git y Github

01

Branching

Se crea una rama específica para una nueva funcionalidad (ej. feature/api-books) para mantener limpio el código estable (main)

02

Staging & Commit

Se seleccionan los archivos modificados (git add) y se guardan con un mensaje descriptivo (git commit -m "Integración de API externa").

03

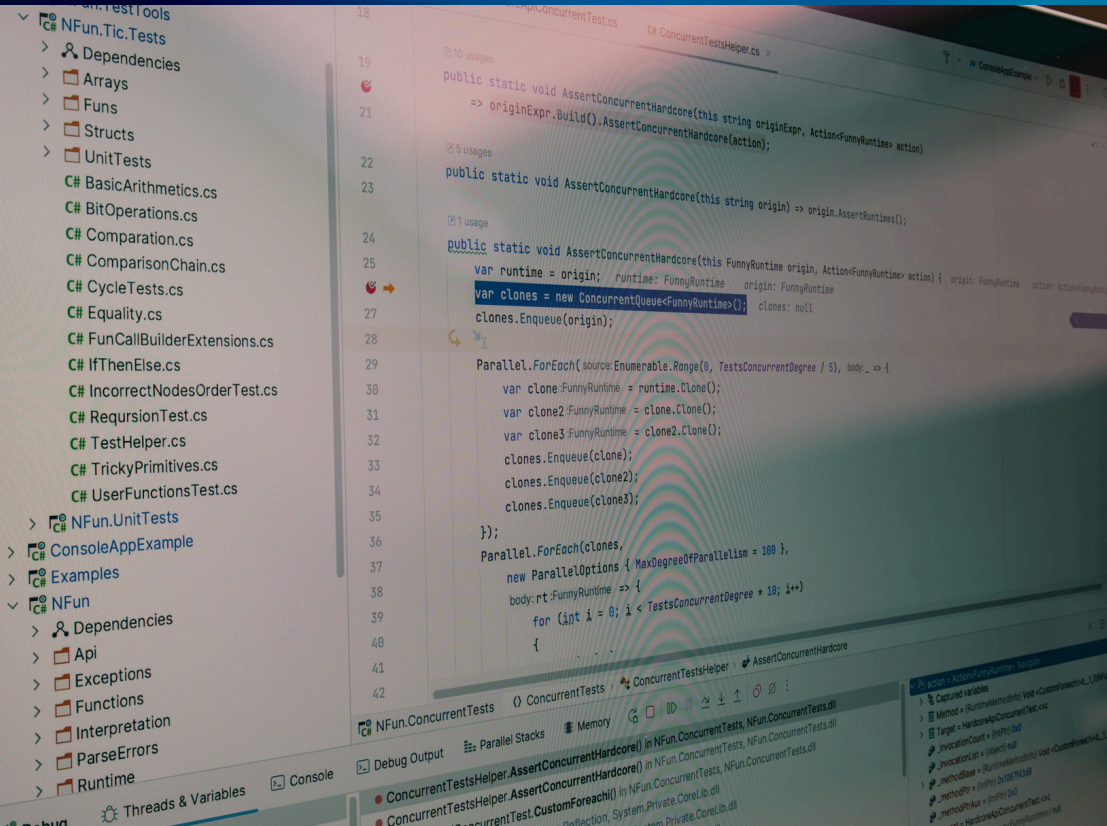
Push & PR

Se sube la rama a GitHub (git push) y se abre un Pull Request (PR) para que el equipo revise el código antes de unirlo al proyecto

04

Merge

Una vez aprobado, el código se fusiona con la rama principal, asegurando que cada cambio esté validado.



Gestión de ramas

01

Ramas utilizadas

- main
- develop

02

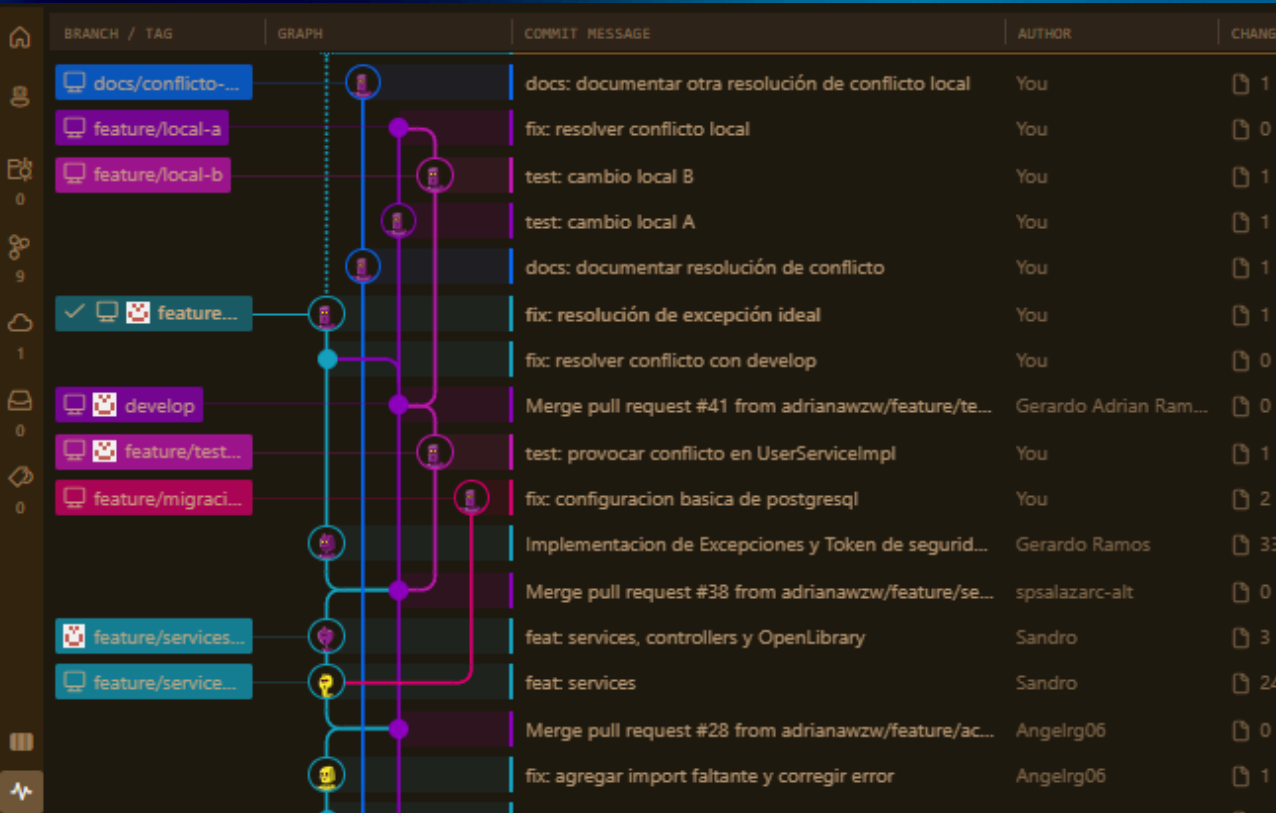
Desarrollo paralelo

Cada integrante trabajó en ramas feature/ independientes para evitar afectar la rama principal.

03

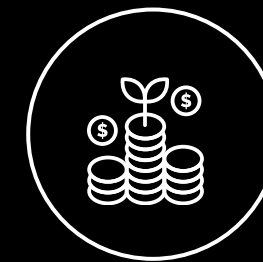
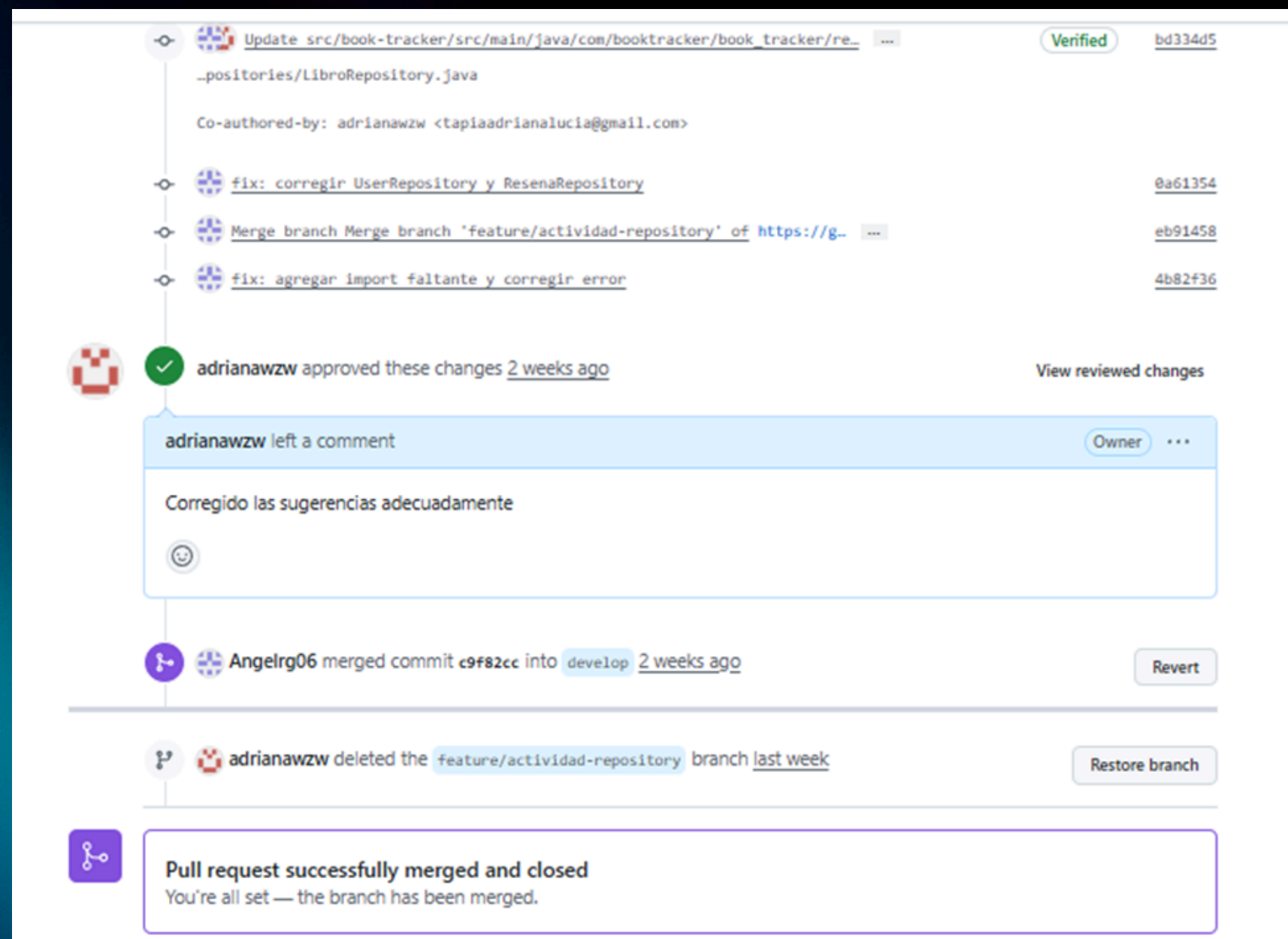
Pull Requests (PR)

Los cambios fueron integrados mediante Pull Requests revisados por el equipo con comentarios descriptivos.



Revisiones, sugerencias y comentarios

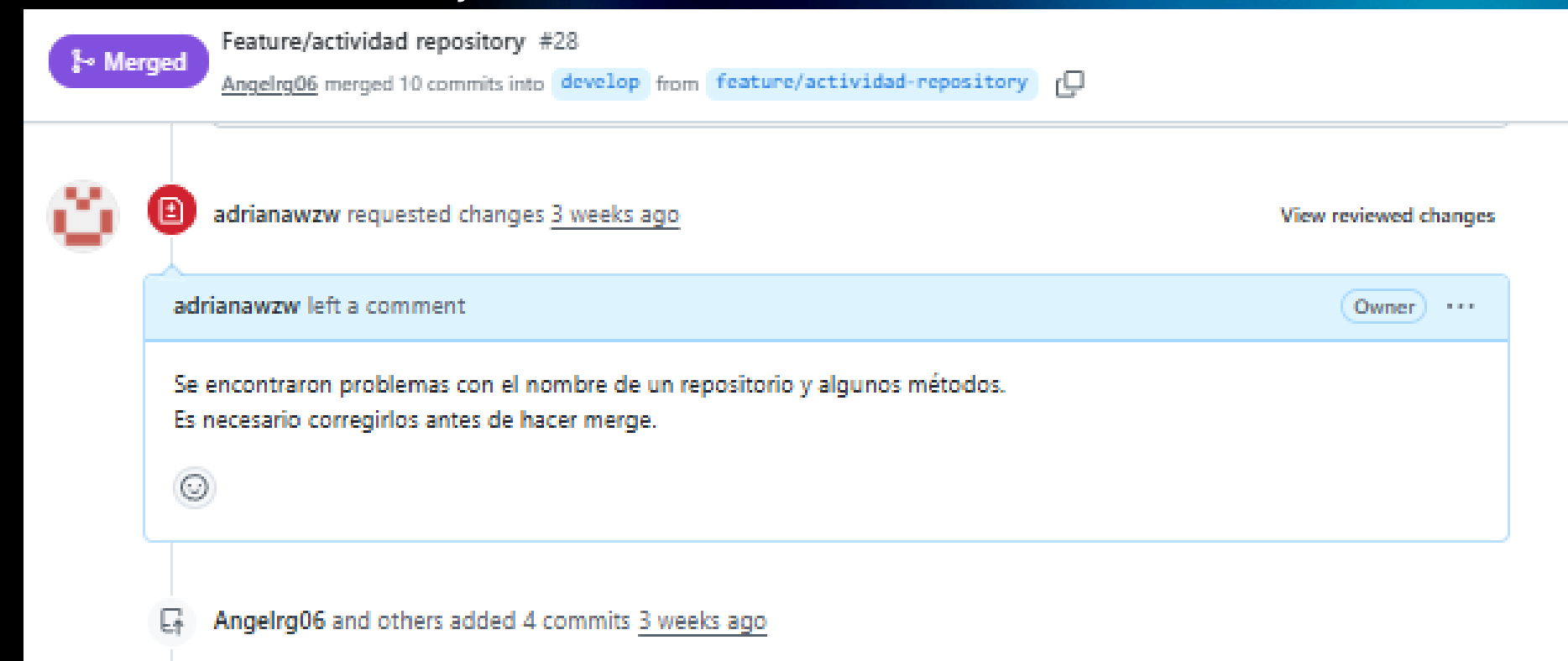
En GitHub, las revisiones de Pull Request (PR) sirven para que otros desarrolladores revisen tu código antes de unirlo (merge) a otra rama, normalmente develop o main.



Revisiones

Cuando alguien revisa el código del PR puede:

- aprobarlo
- pedir cambios
- dejar comentarios



Sugerencias

GitHub permite dejar cambios sugeridos directamente.

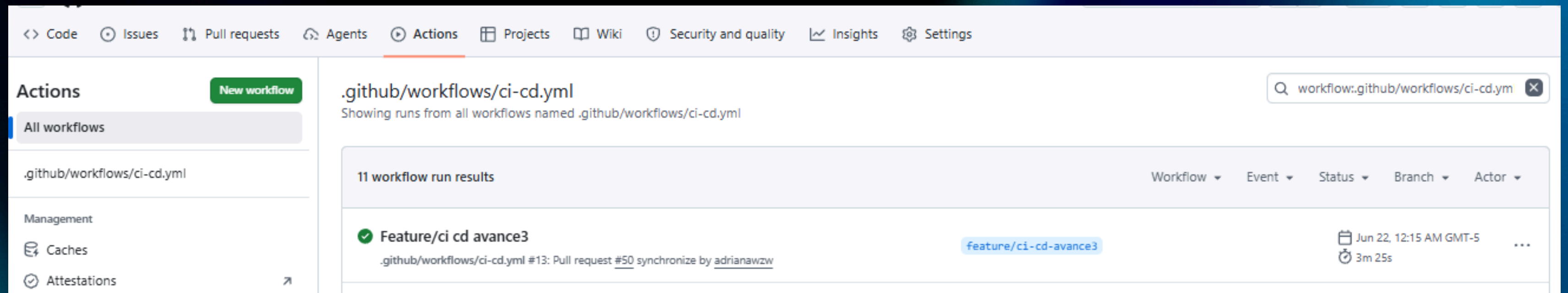


Comentarios

Los comentarios son mensajes que se dejan sobre líneas específicas del código.

Integración continua CI

Es una práctica de desarrollo de software que consiste en integrar de manera frecuente los cambios de los desarrolladores en un repositorio compartido. Su objetivo principal es detectar errores de forma temprana mediante la ejecución automática de procesos de compilación y pruebas.

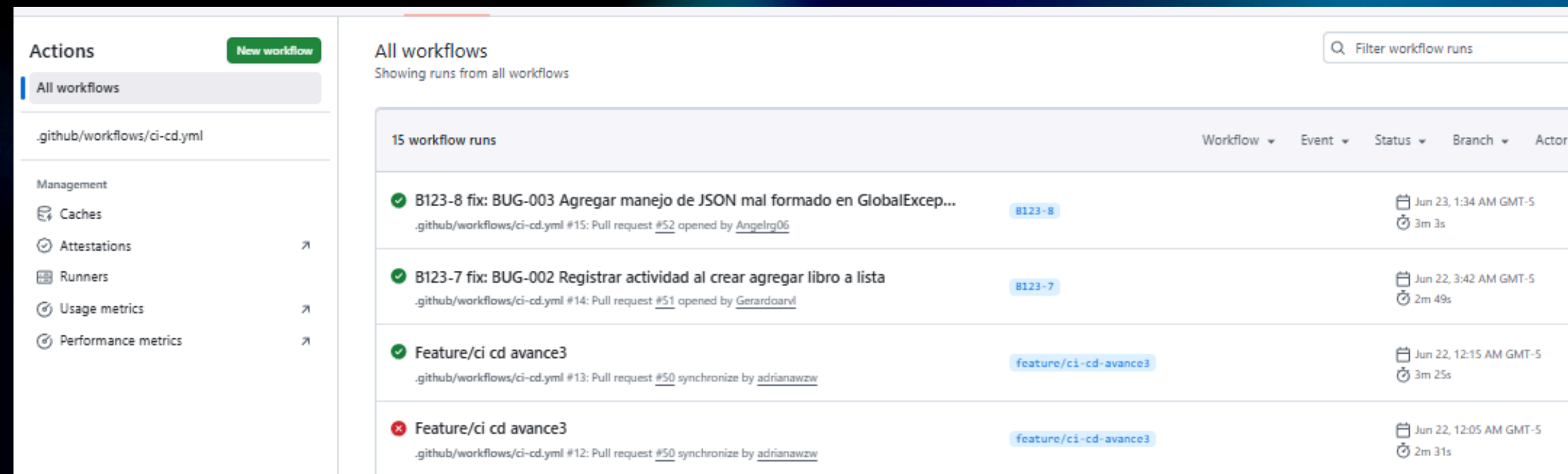
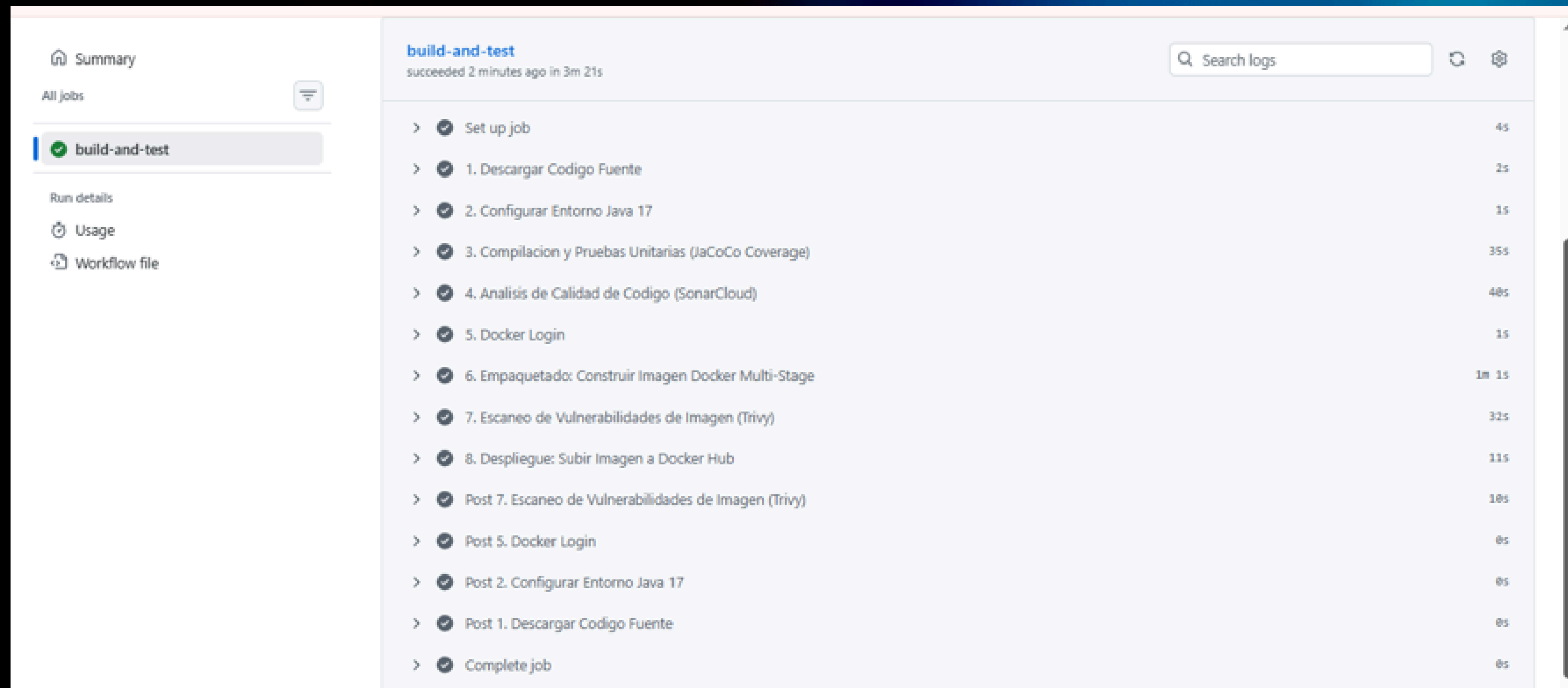


GitHub Actions es la plataforma de automatización que tú configuras (mediante tu archivo .yml) para que actúe como ese revisor incansable.

GitHub Actions Workflow

Es un proceso automatizado y configurable que se ejecuta en un repositorio de GitHub. Es el componente central de GitHub Actions y funciona como un script que le dice a GitHub qué tareas debe realizar de forma automática cuando ocurre un evento específico.

- Configuramos un Workflow en un archivo con formato YAML dentro del directorio `.github/workflows`. Un workflow se compone de elementos clave definidos en código.
- Además de la compilación, posteriormente se integran herramientas de calidad, seguridad y dockerización, permitiendo automatizar gran parte del proceso de validación del proyecto.
- Estructuralmente, el archivo organiza las tareas en bloques llamados jobs;, teniendo como eje central el trabajo de compilación y pruebas.



Calidad y Seguridad

Calidad de Código (SonarCloud)

Es una plataforma de análisis estático de código basada en la nube que evalúa de forma automatizada el código fuente del proyecto para asegurar que cumpla con los estándares de calidad y buenas prácticas de desarrollo.

Snyk y Trivy

Snyk escanea las librerías de terceros cruzándolas con una base de datos pública de vulnerabilidades conocida como CVE (Common Vulnerabilities and Exposures), alertando si el proyecto utiliza código externo desactualizado. Trivy filtra y reporta de forma estricta los fallos de seguridad bajo las clasificaciones

Jacoco

JaCoCo (Java Code Coverage) es una herramienta excelente que sirve exclusivamente para medir la cobertura de código (Code Coverage) en proyectos Java.

The screenshot shows the Snyk web interface. On the left is a sidebar with the Snyk logo and navigation links: Dashboard, Projects, Integrations, Members, and Settings. The main content area displays a 'Security check for adrianawzw/book-tracker' with a 'SUCCESS' status. It lists details for the commit (98ccf4d5), repository (adrianawzw/book-tracker), organization (adrianawzw), and run date (a day ago). Below this, a 'Results' section shows a single item: 'src/book-tracker/pom.xml' with a 'SUCCESS' status and the message 'No manifest changes detected'.

com.booktracker.book_tracker.services.impl													
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxly	Missed	Lines	Missed	Methods	Missed	Classes	
ListServiceImpl		51 %		n/a	5	7	13	24	5	7	0	1	
LibroServiceImpl		32 %		50 %	8	10	19	25	6	7	0	1	
UserServiceImpl		24 %		50 %	9	11	21	27	8	9	0	1	

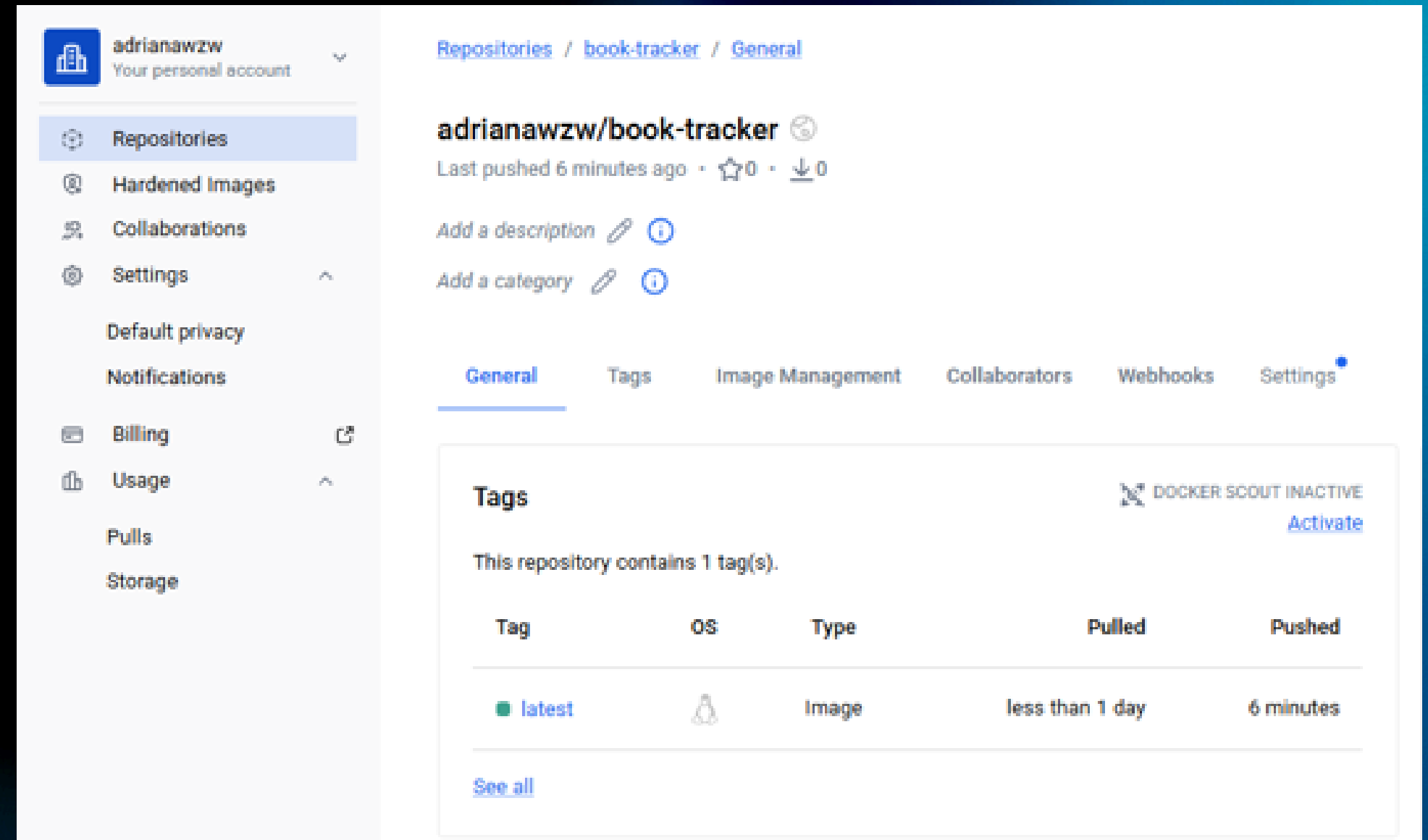
The screenshot shows the Trivy web interface. The top navigation bar includes 'adrianawzw', 'Projects', and 'adrianawzw/book-tracker' with a 'main' branch selector. The main content area displays a security scan for 'src/book-tracker/pom.xml'. It shows a 'C' (Critical) issue for 'org.springframework.boot:spring-boot-devtools@3.5.13' with a 'PRIORITY SCORE (MAX)' of 696. Below this, it lists 'Fixable issues' (2), 'Issues with no supported fix' (0), and 'Vulnerable dependencies' (0). A section titled 'Upgrading to org.springframework.boot:spring-boot-devtools@3.5.14 fixes 2 issues' shows two specific issues: 'Improper Validation of Certificate with Host Mismatch' (CWE-297, CVSS 9.3, 679) and 'Improper Validation of Certificate with Host Mismatch' (CWE-297, CVSS 9.2, 674). At the bottom right, there is a button to 'Upgrade to 3.5.14'.

Dockerización

Dockerfile: Es el archivo de configuración de texto plano que contiene todas las instrucciones secuenciales necesarias para empaquetar la aplicación.

Construcción de Imagen: Es el proceso donde el motor de Docker lee el Dockerfile, descarga las capas base y empaqueta el compilado .jar de la aplicación junto con sus dependencias en una unidad ligera y portable llamada Imagen.

Docker Hub: Es un registro o repositorio público en la nube para las imágenes,



The screenshot shows the Docker Hub interface for the repository 'adrianawzw/book-tracker'. The left sidebar contains the user profile 'adrianawzw' and a navigation menu with options: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area shows the repository details for 'adrianawzw/book-tracker', including the last push time (6 minutes ago), star and download counts (0 each), and options to add a description or category. Below this, there are tabs for General, Tags, Image Management, Collaborators, Webhooks, and Settings. The 'Tags' tab is active, displaying a table of tags. The table has columns for Tag, OS, Type, Pulled, and Pushed. A single tag 'latest' is listed with OS 'linux', Type 'Image', Pulled 'less than 1 day', and Pushed '6 minutes'. A 'See all' link is at the bottom. A 'DOCKER SCOUT INACTIVE' banner with an 'Activate' link is also visible.

Tag	OS	Type	Pulled	Pushed
latest	linux	Image	less than 1 day	6 minutes

Docker Multi-Stage

01

Stage 1 → Maven Build (Etapa de Compilación)

Se utiliza una imagen base pesada que incluye el Kit de Desarrollo de Java (JDK) y Maven (por ejemplo, maven:3.9-eclipse-temurin-17). Su único objetivo es compilar el código fuente y empaquetar el archivo ejecutable app.jar.

02

Stage 2 → JRE Runtime (Etapa de Ejecución)

Se crea una etapa completamente nueva a partir de una imagen extremadamente ligera que solo contiene el Entorno de Ejecución de Java (JRE), como eclipse-temurin:17-jre-alpine. Mediante una instrucción especial, se copia únicamente el archivo app.jar generado en el Stage 1, desechando todo el compilador de Maven y archivos temporales sobrantes.

Dockerfile

```
1  # =====
2  # ETAPA 1: BUILD
3  # =====
4  FROM maven:3.9.9-eclipse-temurin-17 AS builder
5
6  WORKDIR /app
7
8  COPY pom.xml .
9
10 RUN mvn dependency:go-offline
11
12 COPY src ./src
13
14 RUN mvn clean package -DskipTests
15 # =====
16 # ETAPA 2: RUNTIME
17 # =====
18 FROM eclipse-temurin:17-jre
19
20 WORKDIR /app
21
22 COPY --from=builder /app/target/*.jar app.jar
23
24 EXPOSE 8080
25
26 ENTRYPOINT ["java", "-jar", "app.jar"]
```


Pipeline CI/CD

Workflow file for this run

[.github/workflows/ci-cd.yml](#) at 98ccf4d

```
1  name: Integracion Continua BookTracker      39
2                                          40
3  on:                                          41
4    pull_request:                             42
5      branches:                               43
6        - develop                             44
7        - main                                45
8                                          46
9  jobs:                                       47
10    build-and-test:                          48
11      runs-on: ubuntu-latest                 49
12                                          50
13    defaults:                                51
14      run:                                     52
15        working-directory: src/book-tracker  53
16                                          54
17    steps:                                    55
18      - name: 1. DescargarCodigoFuente      56
19        uses: actions/checkout@v4           57
20                                          58
21      - name: 2. ConfigurarEntornoJava17    59
22        uses: actions/setup-java@v4         60
23        with:                               61
24          java-version: '17'                62
25          distribution: 'temurin'           63
26                                          64
27      - name: 3. CompilacionyPruebasUnitarias(JaCoCoCoverage) 65
28        run: mvn clean verify               66
29                                          67
30      - name: 4. AnalisisdeCalidaddeCodigo(SonarCloud)         68
31        env:                                                     69
32          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }               70
33          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }                 71
34        run: |                                                    72
35          mvn sonar:sonar \                                       73
36            -Dsonar.projectKey=adrianawzw_book-tracker \       74
37            -Dsonar.organization=adrianawzw \                  75
38            -Dsonar.token=${ secrets.SONAR_TOKEN }              76
39
40      - name: 5. Docker Login                               77
41        uses: docker/login-action@v3                         78
42        with:                                                 79
43          username: ${ secrets.DOCKER_USERNAME }              80
44          password: ${ secrets.DOCKER_PASSWORD }              81
45
46      - name: "6. Empaquetado: Construir Imagen Docker Multi-Stage" 82
47        run: docker build -t adrianawzw/book-tracker:latest .  83
48
49      - name: 7. Escaneo de Vulnerabilidades de Imagen (Trivy)      84
50        uses: aquasecurity/trivy-action@master                85
51        with:                                                  86
52          image-ref: 'adrianawzw/book-tracker:latest'         87
53          format: 'table'                                       88
54          exit-code: '0' # <-- Cambiado de '1' a '0'           89
55          severity: 'HIGH,CRITICAL'                           90
56
57      - name: "8. Despliegue: Subir Imagen a Docker Hub"          91
58        run: docker push adrianawzw/book-tracker:latest      92
```

Un pipeline es un flujo de trabajo automatizado que conecta una serie de etapas secuenciales para procesar el código del proyecto. Su función principal es tomar cada nuevo cambio introducido por los desarrolladores para compilarlo, probarlo y empaquetarlo automáticamente. Esto garantiza que el software se distribuya de forma rápida, segura y libre de errores humanos en cada actualización

Unificación de Procesos: El pipeline de GitHub Actions integra de manera secuencial 4 pilares fundamentales del desarrollo moderno: Validación, Calidad, Seguridad y Distribución.

Dockerización y Pipeline
CI/CD

Pipeline CI/CD

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

build-and-test

succeeded 2 days ago in 2m 59s

Search logs

1. DescargarCodigoFuente2s

2. ConfigurarEntornoJava170s

1Node 20 is being deprecated. This workflow is running with Node 24 by default. If you need to temporarily use Node 20, you can set the ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true environment variable. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>

2Run actions/setup-java@v4

3with:

4java-version: 17

5distribution: temurin

6java-package: jdk

7check-latest: false

8server-id: github

9server-username: GITHUB_ACTOR

10server-password: GITHUB_TOKEN

11overwrite-settings: true

12job-status: success

13token: ***

14Installed distributions

build-and-test

succeeded 2 days ago in 2m 59s

Search logs

3. Compilacion y Pruebas Unitarias (JaCoCo Coverage)

```
archive, adding nested dependencies in BOOT-INF/.
11012 [INFO] The original artifact has been renamed to /home/runner/work/book-tracker/book-tracker/src/book-tracker/target/book-tracker-jar.original
11013 [INFO]
11014 [INFO] --- jacoco:0.8.11:report (report) @ book-tracker ---
11015 [INFO] Loading execution data file /home/runner/work/book-tracker/book-tracker/src/book-tracker/target/jacoco.exec
11016 [INFO] Analyzed bundle 'book-tracker' with 51 classes
11017 [INFO] -----
11018 [INFO] BUILD SUCCESS
11019 [INFO] -----
11020 [INFO] Total time: 20.473 s
11021 [INFO] Finished at: 2026-06-23T06:35:01Z
11022 [INFO] -----
```

4. Analisis de Calidad de Codigo (SonarCloud)

8. Despliegue: Subir Imagen a Docker Hub

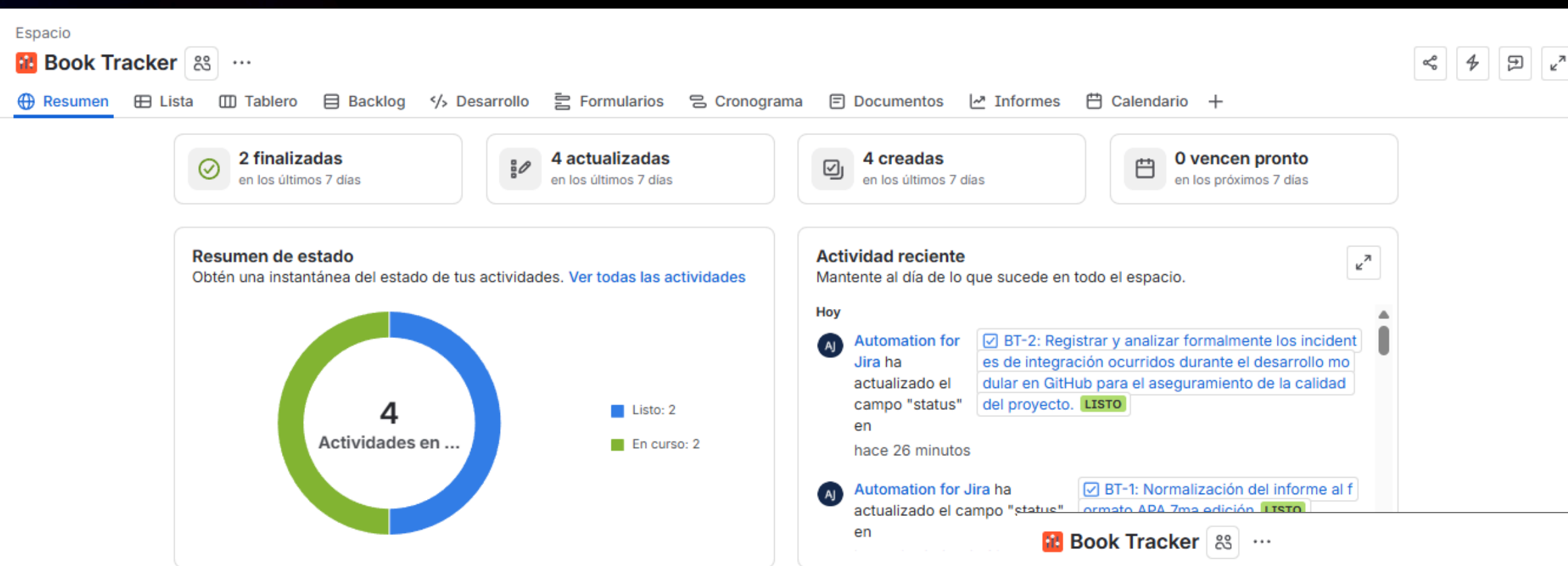
```
1 Run docker push ***/book-tracker:latest
7 The push refers to repository [docker.io/***/book-tracker]
8 9fc1b63be039: Preparing
9 6767df3a16b8: Preparing
10 3606fe3d754b: Preparing
11 9188a8be522a: Preparing
12 a5808d677549: Preparing
13 b18ee137480a: Preparing
14 ab0ae19b58df: Preparing
15 e8c084c1b320: Preparing
16 b18ee137480a: Waiting
17 ab0ae19b58df: Waiting
18 e8c084c1b320: Waiting
19 a5808d677549: Layer already exists
20 9188a8be522a: Layer already exists
21 3606fe3d754b: Layer already exists
22 ab0ae19b58df: Layer already exists
23 b18ee137480a: Layer already exists
24 e8c084c1b320: Layer already exists
25 6767df3a16b8: Pushed
26 9fc1b63be039: Pushed
27 latest: digest: sha256:40472bd4ef87a725e082c924b206584007724e55102e708d4d0eb9dacf3be766 size: 1993
```


Jira

Gestión de proyectos con JIRA

Se utilizaron herramientas diferenciadas:

- Jira: Planificación general, asignación de responsabilidades, seguimiento de actividades globales.
- GitHub Projects e Issues: Tareas específicas de programación e integración de código.



Book Tracker

Buscar en el ba... [SR](#) [AT](#) [GV](#) [S](#) [Filtro](#)

☐ **Sprint1 - Avance de Proyecto 2** 4 may – 3 jun (4 actividades)

Completar la documentación técnica del Avance 1, incluyendo la normalización APA, el reporte de conflictos, la especificación de requisitos y el diccionario de datos de la base de datos local.

☒ BT-1	Normalización del informe al formato APA 7ma edición	LISTO	-	=	AT
☒ BT-2	Registrar y analizar formalmente los incidentes de integración ocurridos durante el desarrollo modular en GitHub para el aseguram...	LISTO	-	=	SR
☒ BT-3	Especificación de requisitos funcionales y no funcionales del backend	EN CURSO	-	=	GV
☒ BT-4	Elaboración del diccionario de datos y reglas de integridad de la BD	EN CURSO	-	=	S

+ Crear

☐ **Backlog** (0 actividades)

0 0 0 [Crear sprint](#)

Tu backlog está vacío.

Jira Service Management ITSM e Incidentes

La gestión de incidentes se realiza mediante la herramienta Jira Service Management, la cual permite administrar solicitudes, errores y eventos que afectan el correcto funcionamiento del sistema, facilitando su seguimiento y resolución dentro del equipo de desarrollo.

01

ITSM

ITSM es un enfoque de gestión de servicios de TI que se centra en la planificación, entrega, operación y mejora continua de los servicios tecnológicos.

02

Incidente

Un incidente es cualquier interrupción no planificada o degradación de un servicio que afecta el funcionamiento normal del sistema.

03

Creación del proyecto

Para la gestión de incidencias se creó el proyecto en Jira Service Management denominado Book Tracker ITSM. Este proyecto permite centralizar el registro, seguimiento y resolución de los incidentes y bugs del sistema

Incidentes y Bugs

Jira Service Management

Search

+ Create

11 days left

Ask Rovo

Spaces / Book Tracker Service Desk

All work

Search w... AT AC GV SR Filter 1 Group

☐	Work	Assignee	Reporter	Priority	Status
☐	B123-9 La librería de API OpenLib...	ALUMNO - SANDRO PIERO SALAZAR CA...	Adriana Tapia	Medium	OPEN
☐	B123-8 Error 500 en /auth/autenti...	Samir Rojas	Gerardo Adrian Ra...	Medium	COMPLETED
☐	B123-7 Al ejecutar el endpoint de ...	Gerardo Adrian Ramos Valdivia	Adriana Tapia	Medium	CLOSED
☐	B123-6 No se registra actividad C...	Adriana Tapia	Gerardo Adrian Ra...	Medium	CLOSED

INC-001: Error PostgreSQL

Incidente relacionado con fallos en la conexión o ejecución de consultas en la base de datos PostgreSQL.

BUG-001: No registra CREO_LISTA

Error en la funcionalidad de creación de listas de libros, donde no se persisten los datos correctamente.

BUG-002: No registra AGREGO_LIBRO

Problema en la funcionalidad de guardado de libros en listas que impide el registro en actividades cuando un libro es guardado en una lista.

Automatización

El flujo de gestión de incidencias en Jira Service Management fue automatizado para optimizar el seguimiento del ciclo de vida de cada ticket, reduciendo la intervención manual y asegurando la trazabilidad del proceso.

Regla 1: PR Mergeado
→ Resuelto



Adriana
Tapia



Global

7 days ago



Regla 2: PR Creado →
En desarrollo



Adriana
Tapia



Global

7 days ago



⚡ Regla 1: PR Mergeado → Resuelto



Pull request merged

Flow is run when a pull request is merged



Transition the work item to

COMPLETED

Automatización Jira-GitHub

Integración Jira-GitHub

La integración entre Jira y GitHub permite establecer una trazabilidad completa entre los tickets de incidencias y el desarrollo del código fuente, conectando directamente la gestión de tareas con el repositorio del proyecto.

Ticket Jira->Branch->Commit->Pull Request->Merge

The screenshot displays a Jira ticket titled "No se registra actividad CREO_LISTA al crear una lista de lectura" (B123-6) and its corresponding GitHub Pull Request. The Jira ticket details the issue, expected behavior (registering activity), and actual behavior (activity not appearing). The GitHub Pull Request, titled "B123-6 fix: BUG-001 Registrar actividad al crear listas #48", shows the code changes made to resolve the issue, including the implementation of the "ActividadRepository" and the resulting partial fix where transactions are now recorded in the activity table. The workflow is summarized as: Ticket Jira->Branch->Commit->Pull Request->Merge.

Spaces / Book Tracker Service... / B123-6

No se registra actividad CREO_LISTA al crear una lista de lectura

Create subtask Link work item Create ...

GV Gerardo Adrian Ramos Valdivia raised this request via Jira [View request in portal](#) [Hide details](#)

Description
Al ejecutar el endpoint de creación de listas, la lista se almacena correctamente en la base de datos, pero no se genera el registro correspondiente en la tabla Actividades.

***Comportamiento esperado:** Registrar actividad CREO_LISTA.
***Comportamiento actual:** La actividad no aparece en el historial.

Affected hardware None

Urgency None

Impact

B123-6 fix: BUG-001 Registrar actividad al crear listas #48 [edit](#)

Merged [adrianawzw](#) merged 1 commit into [develop](#) from [B123-6](#) 4 days ago

Conversation 1 Commits 1 Checks 0 Files changed 1

[adrianawzw](#) commented 5 days ago [Owner](#) ...

Descripción
Se resolvió el BUG reportado B123-6 No se registra actividad CREO_LISTA al crear una lista de lectura

Cambios realizados
Se utilizó el repository de actividad en el método de la creación de la lista.
Implementación de repository:
ActividadRepository

Resultado parcial
Ahora se registra también la transacción en la tabla actividad

Reviewers
Angelrg06
Gerardoarvl

Assignees
No one—[assign yourself](#)

Labels
None yet

Projects
None yet

Milestone
No milestone

ced7d03

Conclusiones

la utilización de Jira Service Management permitió gestionar incidentes, errores y solicitudes de manera estructurada, mientras que la integración con GitHub facilitó la automatización de flujos de trabajo y el seguimiento de las actividades de desarrollo. En conjunto, la implementación de prácticas de CI/CD, DevOps y gestión de servicios de TI contribuyó significativamente a mejorar la calidad, seguridad, mantenibilidad y organización del proyecto Book Tracker.